CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
ENGENHARIA ELÉTRICA

CHRISTOPHER CÉSAR DE MELO ANDRADE
RÚBIA FERNANDES DE ANDRADE

LABORATÓRIO DE SISTEMAS MICROPROCESSADOS
RELATÓRIO: PRÁTICA 2

Belo Horizonte, 11 de Setembro de 2025

Consideremos, para esse relatório e demais relatórios, ***Vx*** referente aos valores literais da variável ***x*** e ***Fx*** o valor do endereço na memória de programa.

1. Program ex01.asm

```
    #include <p18f4550.inc>

    #define VA 0xF8
    #define VB 0x08
    #define VC 0x80
    #define VD 0xFF
    #define VE 0x00
    #define VF 0xFF
    #define VG 0xF6
    #define VH 0x0A

    org 0x0000
    bra    start

    org 0x0020
start:
    ; 1) a. CARRY / DIGITAL CARRY
    ; 0xF8 + 0x08 = 0x00
    ; (-8) + (8)  = 0
    MOVLW 0x00
    MOVWF STATUS, 0 ; Limpar STATUS para próxima operação
    MOVLW VA
    ADDLW VB
    ; 2) b. OVERFLOW
    ; 0x80 + 0xFF = 0x7F
    ; (-127 + (-1) = (-128) -> overflow -> (127)
    MOVLW 0x00
    MOVWF STATUS, 0 ; Limpar STATUS para próxima operação
    MOVLW VC
    ADDLW VD
    ; 3) c. NEGATIVO
    ; 0x00 + 0xFF = 0xFF
    ; (0) + (-1) = (-1)
    MOVLW 0x00
    MOVWF STATUS, 0 ; Limpar STATUS para próxima operação
    MOVLW VE
    ADDLW VF
    ; 4) d. ZERO
    ; 0xF6 + 0x0A = 0x00
    ; (-10) + (10) = (0)
    MOVLW 0x00
    MOVWF STATUS, 0 ; Limpar STATUS para próxima operação
    MOVLW VG
    ADDLW VH

    bra    start
    end
```

Para esse exercício de soma de literal com acumulador, consideremos todos os números denotados em complemento de 2:

a) Para ativar as *flags* de *CARRY* e *DIGITAL CARRY* simultâneamente, consideramos somar -8 (*0xF8*) e 8 (*0x08*)

$$\begin{array}{rl} & 11111 \ \leftarrow \text{carrys} \\ & 00001000 \\ + & 11111000 \\ \hline & 100000000 \end{array}$$

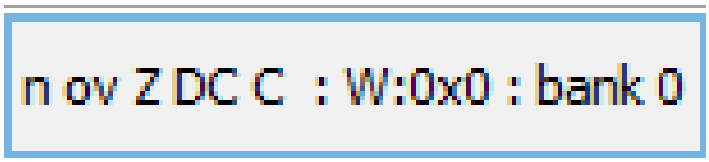

Figura 1:. Registradores Especiais após operação (0xF8 + 0x08)

Nesse exemplo, tanto o *CARRY* quanto o *DIGITAL* são ativados, pois a operação tem *carrys* do 5º ao 1º *bit* mais significativo.

Como todos os 8 bits são 0, a flag *ZERO* também foi ativada.

b) Consideramos somar o número negativo que está na extremidade do intervalo [2^7, -2^7], -127 (*0x80*) com -1 (*0xFF*) para quebrar a escala de gerar um *OVERFLOW*.

$$\begin{array}{rl} & 1 \\ & 10000000 \\ + & 11111111 \\ \hline & 01111111 \end{array}$$

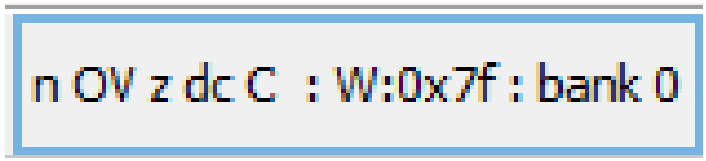

Figura 2:. Registradores Especiais após operação (0x80 + 0xFF)

Após quebrar a escala, o *WREG* passou de -127 para +127, ativando a *flag* de *OVERFLOW*.

c) Nesse exemplo, consideramos somar 0 (*0x00*) com -1 (*0xFF*), para resultar um número negativo e ativar a *flag* NEGATIVE

$$\begin{array}{rl} & 00000000 \\ + & 11111111 \\ \hline & 11111111 \end{array} \quad \leftarrow \textit{bit de sinal é 1}$$

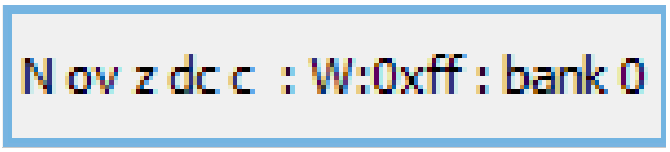

Figura 3:. Registradores Especiais após operação (0x00 + 0xFF)

Como o resultado da operação tem o primeiro *bit* (sinal) igual a 1, a *flag NEGATIVE* é ativada.

d) Consideramos a operação -10 (*0xF6*) + 10 (*0x0A*) para testar a *flag ZERO*

$$\begin{array}{r} 11110110 \\ +\ 00001010 \\ \hline 00000000 \end{array} \quad <- \textit{ todos os bits são 0}$$

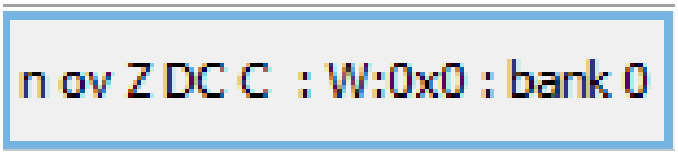

Figura 4:. Registradores Especiais após operação (0xF6 + 0x0A)

A *flag ZERO* é ativada, pois todos os *bits* são 0

2. Programa ex02.asm

```
    #include <p18f4550.inc>

    ; Endereços de Memória  (A e B)
    #define FA 0x00
    #define FB 0x01
    ; Valores literais      (a e b)
    #define VA 0x01
    #define VB 0xEF

    org 0x0000
    bra    start

    org 0x0020
start:
    ; VB = VB + VA (VB += VA)
    ; 0xEF + 0x01 = 0xF0

    MOVLW VB    ; Carregar VB em FB
    MOVWF FB, 0
    MOVLW VA    ; Carregar VA em FA
    MOVWF FA, 0
    ADDWF FB, F ; Somar VA + VB e guardar resultado em VB

    bra    start
    end
```

Para esse programa, consideremos utilizar os valores **VA** e **VB** iguais a *0x01* e *0xEF*, respectivamente. A operação aritmética, *VB = VB + VA*, podem ser interpretadas nos dois casos de representações, com e sem sinal, pois *0x01* equivale a (1) e *0xEF* equivale a (239) em binário e (-17) em complemento de 2.

Levando em conta essas informações, sabe-se que o resultado, ao final do programa, deve ser 240 (1 + 239) em binário e -16 (-17 + 1) em complemento de 2, equivalente ao hexadecimal *0xF0*.

Verifiquemos:

Primeiro carregamos os literais de **VA** e **VB** no *WREG* e depois passamos para os respectivos endereços **FA** e **FB**, em seguida a instrução ***ADDWF*** soma o valor atual do ***WREG*** (**VA**) com o valor no endereço **FB** (**VB**). O resultado dessa soma é salvo no próprio endereço, pois a operação é do tipo *VB += VA*.

Ao final, os seguintes valores na memória de dados são mostrados na Figura 5:

Address	00	01	02	03	04
000	01	F0	00	00	00
010	00	00	00	00	00
020	00	00	00	00	00
030	00	00	00	00	00
040	00	00	00	00	00
050	00	00	00	00	00

Figura 5:. Endereço FB (0x01), destacado em azul, com valor 0xF0 como esperado

3. Programa ex03.asm

```
#include <p18f4550.inc>

#define FAL 0x01
#define FAH 0x00
#define FBL 0x03
#define FBH 0x02

#define VAL 0x01    ; 0x0001
#define VAH 0x00
#define VBL 0xFA    ; 0x00FA
#define VBH 0x00

org 0x0000
MOVLW VBL
MOVWF FBL, 0
MOVLW VBH
MOVWF FBH, 0     ; Carrega VB em FB
MOVLW VAL
MOVWF FAL, 0
MOVLW VAH
MOVWF FAH, 0     ; Carrega VA em FA
bra    start
```

```
    org 0x0020
start:
    MOVF FAL, W
    ADDWF FBL, F, 0  ; Soma da parte baixa

    MOVF FAH, W
    ADDWFC FBH, F, 0 ; Soma da parte alta (VA + VB + CARRY)

    bra    start
    end
```

Esse programa é similar ao *ex02.asm* em partes. Nesse caso, consideremos utilizar 2 *bytes* para armazenar um número de 16 *bits*, assim denotamos **VxL** como o valor da parte baixa de **x** e **VxH** como o valor da parte alta de **x**, além disso, **FxL** e **FxH** são os endereços de memória que armazenam a parte baixa e alta, respectivamente de **x.**

Antes de executar a *label 'start'*, carregamos **VA** e **VB** na memória, a escolha disso se dá pela necessidade de observar o momento em que o bit *CARRY* do Registrador *STATUS* é setado, atualizando a parte alta do número de 16 *bits*.

Para esse programa **VA** é igual a *0x0001* e **VB**, *0x00FA*. Então, esperasse que depois de 6 ciclos de execução da *label 'start'* o valor de **VB** seja igual a *0x0100*, isto é, incremente a parte alta e zere a parte baixa.

Ao fim dos 6 ciclos, os seguintes valores na memória de dados são mostrados na Figura 6:

Address	00	01	02	03	04
000	00	01	01	00	00
010	00	00	00	00	00
020	00	00	00	00	00
030	00	00	00	00	00
040	00	00	00	00	00
050	00	00	00	00	00

Figura 6:. Endereços FBH e FBL destacados em azul, com valor concatenado de 0x0100 como esperado

4. Programa ex04.asm

```
    #include <p18f4550.inc>

    #define VA 0x4A
    #define VB 0xA5
    #define FA 0x00
    #define FB 0x01
    #define FC 0x02

    org 0x0000
    bra    start

    org 0x0020
start:

    ; Carregar variáveis
    MOVLW VA
    MOVWF FA, 0
    MOVLW VB
    MOVWF FB, 0

    MOVLW 0x0F      ; Mascara do nibble menos significativo
    ANDWF FA, W     ; Filtrage de bits
    MOVWF FC, 0     ; Resultado é jogado no endereço FC

    MOVLW 0xF0      ; Mascara do nibble mais significativo
    ANDWF FB, W     ; Filtrage de bits
    IORWF FC, F, 0  ; Concatenação de resultados no endereço FC

    bra    start
    end
```

Consideremos para esse programa os valores *0x4A* e *0xA5* referentes às variáveis **A** e **B** respectivamente. Para os valores considerados, o programa deve retornar um valor no endereço **FC** igual a *0xAA*.

Depois de carregar as variáveis as seguintes operações lógicas foram realizadas:

0x4A = *0b01001000*

Selecionando *nibble* menos significativo de **A:**

$$\begin{array}{ll} & 0b01001000 \\ AND & 0b00001111 \\ \hline & 0b00001000 \end{array}$$

<- *Low Nibble Mask* (*0x0F*)

= *0x0A*

Após a operação, para não alterar o valor de **VA** na memória, salvamos o valor filtrado em **FC.**

0xA5 = 0b10000101

Selecionando *nibble* menos significativo de **B:**

$$\begin{array}{ll} & 0b10000101 \\ AND & 0b11110000 \\ \hline & 0b10000000 \end{array}$$

<- *High Nibble Mask* (*0xF0*)

= *0xA0*

O resultado da operação é salvo no *WREG*, pois será utilizado na próxima instrução.

Para concatenar os *nibbles* selecionados utilizamos a instrução ***IORWF.***

$$\begin{array}{ll} & 0b00001000 \\ OR & 0b10000000 \\ \hline & 0b10001000 \end{array}$$

= *0xAA*

IORWF, compara *WREG* com **FC** e armazenamos o resultado em **FC.**

Ao final do programa, os seguintes valores na memória de dados são mostrados na Figura 7:

Address	00	01	02	03	04
000	4A	A5	AA	00	00
010	00	00	00	00	00
020	00	00	00	00	00
030	00	00	00	00	00
040	00	00	00	00	00
050	00	00	00	00	00

Figura 7:. Endereços FC (0x02) destacado em azul, com valor 0xAA como esperado

5. Programa ex05.asm

```
    #include <p18f4550.inc>

    #define VA 0x0A ; VA = 10
    #define VB 0x10 ; VB = 16
    #define FA 0x00
    #define FB 0x01
    #define FC 0x02

    org 0x0000
    bra start
    org 0x0020

start:

    ; Carregando variáveis
    MOVLW VA
    MOVWF FA, 0
    MOVLW VB
    MOVWF FB, 0

    RRCF FB, F       ; FB = FB / 2
    RRCF FB, F       ; FB = FB / 2
    RLCF FA, F, 0    ; FA = FA * 2
    MOVF FA, W       ; W <- FA
    ADDWF FB, W, 0   ; W = FA + FB
    MOVWF FC, 0      ; FC <- W

    bra    start
    end
```

Rotacionar um *bit* para esquerda equivale a multiplicar o número por 2, de forma análoga, rotacionar um *bit* para direita equivale a dividir o número por 2. Seguindo essa ideia, devemos rotacionar **VB** duas vezes para a direita (***RRCF***) e rotacionar **VA** uma vez para a esquerda (***RLCF***), ao final, somar os valores e armazenar no endereço de resultado.

Os valores *0x0A* (10) e *0x10* (16) foram utilizados para **A** e **B**, respectivamente, assim, o resultado final *VC = (10 * 2) + (16 / 4)*, deve ser 24 (*0x18*).

Ao final do programa, os seguintes valores na memória de dados são mostrados na Figura 8:

Address	00	01	02	03	04
000	14	04	18	00	00
010	00	00	00	00	00
020	00	00	00	00	00
030	00	00	00	00	00

Figura 8:. Endereços FC (0x02) destacado em azul, com valor 0x18 como esperado